1. White Box Testing

1.2 Code Quality Analysis

main.py

Initial Pylint Output:

pylint main.py

********** Module main

main.py:1:0: C0114: Missing module docstring (missing-module-docstring)

main.py:4:0: C0116: Missing function or method docstring (missing-function-docstring)

main.py:11:0: C0116: Missing function or method docstring (missing-function-docstring)

Your code has been rated at 8.24/10 (previous run: 8.24/10, +0.00)

- Iteration 1: Added Module level docstring, to fix the error `main.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

'''Entry point for the game'''

from moneypoly.game import Game

- Iteration 2: Added Function docstring to explain what the function does, to fix the error `main.py:4:0: C0116: Missing function or method docstring (missing-function-docstring)`

def get_player_names():

  '''Prompt the user to enter player names and return a list of names'''

- Iteration 3: Added Function docstring to explain what the function does, to fix the error `main.py:11:0: C0116: Missing function or method docstring (missing-function-docstring)`

def main():

    '''Entry point for the game, creates and runs the game with user-provided player names'''

bank.py

Initial Pylint Output:

pylint moneypoly/bank.py

********** Module bank

moneypoly/bank.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/bank.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)

moneypoly/bank.py:5:0: C0115: Missing class docstring (missing-class-docstring)

moneypoly/bank.py:1:0: W0611: Unused import math (unused-import)

Your code has been rated at 7.71/10 (previous run: 7.71/10, +0.00)

- Iteration 4: Removed the unused math import to resolve `moneypoly/bank.py:1:0: W0611: Unused import math (unused-import)`

- Iteration 5: Added a module docstring to resolve `moneypoly/bank.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

'''manages the game's cash reserves, loans, and collections'''

from moneypoly.config import BANK_STARTING_FUNDS

- Iteration 6: Changed absolute import with relative to fix `moneypoly/bank.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)`

from config import BANK_STARTING_FUNDS

- Iteration 7: Added a class docstring to resolve `moneypoly/bank.py:5:0: C0115: Missing class docstring (missing-class-docstring)`

class Bank:

'''Represent the bank: gets balance, collects funds, pays out, and issues loans.'''

board.py

Initial Pylint Output:

pylint moneypoly/board.py

********** Module board

moneypoly/board.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/board.py:1:0: E0401: Unable to import 'moneypoly.property' (import-error)

moneypoly/board.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)

moneypoly/board.py:108:11: C0121: Comparison 'prop.is_mortgaged == True' should be 'prop.is_mortgaged is True' if checking for the singleton value True, or 'prop.is_mortgaged' if testing for truthiness (singleton-comparison)

- Iteration 8: Add a module docstring to resolve `moneypoly/board.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

'''defines the game board, spaces, and helpers'''

from moneypoly.property import Property, PropertyGroup

- Iteration 9: changed absolute import to relative import for `moneypoly/board.py:1:0: E0401: Unable to import 'moneypoly.property' (import-error)`

from property import Property, PropertyGroup

- Iteration 10: changed absolute import to relative import for `moneypoly/board.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)`

from config import (

JAIL_POSITION,

GO_TO_JAIL_POSITION,

FREE_PARKING_POSITION,

```
    INCOME_TAX_POSITION,

    LUXURY_TAX_POSITION,

)
```

- Iteration 11: removed `== True` in the if statement to fix `moneypoly/board.py:110:11: C0121: Comparison 'prop.is_mortgaged == True' should be 'prop.is_mortgaged is True' if checking for the singleton value True, or 'prop.is_mortgaged' if testing for truthiness (singleton-comparison)`

```
if prop.is_mortgaged:

    return False
```

cards.py

Initial Pylint Output:

```
pylint moneypoly/cards.py

************* Module cards

moneypoly/cards.py:4:0: C0301: Line too long (111/100) (line-too-long)

moneypoly/cards.py:5:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:6:0: C0301: Line too long (111/100) (line-too-long)

moneypoly/cards.py:7:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:8:0: C0301: Line too long (113/100) (line-too-long)

moneypoly/cards.py:9:0: C0301: Line too long (113/100) (line-too-long)

moneypoly/cards.py:10:0: C0301: Line too long (113/100) (line-too-long)

moneypoly/cards.py:11:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:12:0: C0301: Line too long (111/100) (line-too-long)

moneypoly/cards.py:13:0: C0301: Line too long (112/100) (line-too-long)
```

moneypoly/cards.py:14:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:15:0: C0301: Line too long (120/100) (line-too-long)

moneypoly/cards.py:19:0: C0301: Line too long (113/100) (line-too-long)

moneypoly/cards.py:20:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:21:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:22:0: C0301: Line too long (111/100) (line-too-long)

moneypoly/cards.py:23:0: C0301: Line too long (113/100) (line-too-long)

moneypoly/cards.py:24:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:25:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:26:0: C0301: Line too long (113/100) (line-too-long)

moneypoly/cards.py:27:0: C0301: Line too long (113/100) (line-too-long)

moneypoly/cards.py:28:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:29:0: C0301: Line too long (112/100) (line-too-long)

moneypoly/cards.py:30:0: C0301: Line too long (111/100) (line-too-long)

moneypoly/cards.py:1:0: C0114: Missing module docstring (missing-module-docstring)

Your code has been rated at 0.38/10 (previous run: 0.38/10, +0.00)

- Iteration 12-35: Make line length shorter to fix error `C0301: Line too long (111/100) (line-too-long)`

{"description": "Advance to Go. Collect $200.",

 "action": "move_to", "value": 0},

- Iteration 36: added module docstring for cards.py to fix error `moneypoly/cards.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

'''Cards module: defines Chance and Community Chest card data and the CardDeck helper'''

import random

config.py

Initial Pylint Output:

pylint moneypoly/config.py

********** Module config

moneypoly/config.py:1:0: C0114: Missing module docstring (missing-module-docstring)

Your code has been rated at 9.29/10 (previous run: 9.29/10, +0.00)

- Iteration 37: added module level docstring to fix error `moneypoly/config.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

'''Config module: defines game constants and configuration parameters for Moneypoly'''

dice.py

Initial Pylint Error:

pylint moneypoly/dice.py

********** Module dice

moneypoly/dice.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/dice.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)

moneypoly/dice.py:25:12: W0201: Attribute 'doubles_streak' defined outside __init__ (attribute-defined-outside-init)

moneypoly/dice.py:2:0: W0611: Unused BOARD_SIZE imported from moneypoly.config (unused-import)

Your code has been rated at 7.04/10 (previous run: 7.04/10, +0.00)

- Iteration 38: added module level docstring to fix error `moneypoly/dice.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

'''Dice class for MoneyPoly game, simulating a pair of six-sided dice with consecutive-doubles tracking.'''

import random

- Iteration 39: remove config module import, as the variable was not being used anywhere to fix `moneypoly/dice.py:2:0: W0611: Unused BOARD_SIZE imported from moneypoly.config (unused-import)` and indirectly `moneypoly/dice.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)` error

- Iteration 40: initialise the attribute `doubles_streak` in `__init__()`, to fix the error `moneypoly/dice.py:25:12: W0201: Attribute 'doubles_streak' defined outside __init__ (attribute-defined-outside-init)`

def __init__(self):

    self.die1 = 0

    self.die2 = 0

    self.doubles_streak = 0

    self.reset()

ui.py

Initial Pylint Output:

pylint moneypoly/ui.py

********** Module ui

moneypoly/ui.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/ui.py:69:4: W0702: No exception type(s) specified (bare-except)

Your code has been rated at 9.55/10 (previous run: 9.55/10, +0.00)

- Iteration 41: added module level docstring to fix error `moneypoly/ui.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

'''UI: functions for printing game status and prompting user input.'''

- Iteration 42: Replace bare except with specific exceptions to avoid catching BaseException and fix `moneypoly/ui.py:69:4: W0702: No exception type(s) specified (bare-except)`

```
    try:
        return int(input(prompt))
    except (ValueError, TypeError, EOFError):
        return default
```

property.py

Initial Pylint Output:

pylint moneypoly/property.py

********** Module property

moneypoly/property.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/property.py:1:0: R0902: Too many instance attributes (9/7) (too-many-instance-attributes)

moneypoly/property.py:6:4: R0913: Too many arguments (6/5) (too-many-arguments)

moneypoly/property.py:48:8: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)

moneypoly/property.py:64:0: C0115: Missing class docstring (missing-class-docstring)

Your code has been rated at 9.15/10

- Iteration 43: Added module level docstring to fix `moneypoly/property.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

"""Property and property-group models for the MoneyPoly board."""

- Iteration 44: Reduced `Property` instance attributes by making `mortgage_value` computed and keeping `houses` as a class default to fix `moneypoly/property.py:3:0: R0902: Too many instance attributes (9/7) (too-many-instance-attributes)`

```
class Property:

    FULL_GROUP_MULTIPLIER = 2

    houses = 0

    @property

    def mortgage_value(self):

        return self.price // 2
```

- Iteration 45: Refactored `Property.__init__` to keep four explicit parameters and parse optional `group` via varargs/kwargs, fixing `moneypoly/property.py:9:4: R0913: Too many arguments (6/5) (too-many-arguments)`

```
def __init__(self, name, position, price, base_rent, *group_args, **kwargs):

    group_from_positional = group_args[0] if group_args else None

    group_from_keyword = kwargs.pop("group", None)
```

- Iteration 46: Removed unnecessary `else` after early return in `unmortgage()` to fix `moneypoly/property.py:65:8: R1705: Unnecessary "else" after "return" (no-else-return)`

```
if not self.is_mortgaged:

    return 0
```

cost = int(self.mortgage_value * 1.1)

- Iteration 47: Added `PropertyGroup` class docstring to fix `moneypoly/property.py:80:0: C0115: Missing class docstring (missing-class-docstring)`

class PropertyGroup:

"""Represents a color group and the properties associated with it."""

player.py

Initial Pylint Output:

pylint moneypoly/player.py

********** Module player

moneypoly/player.py:87:0: C0304: Final newline missing (missing-final-newline)

moneypoly/player.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/player.py:2:0: E0401: Unable to import 'moneypoly.config' (import-error)

moneypoly/player.py:5:0: R0902: Too many instance attributes (8/7) (too-many-instance-attributes)

moneypoly/player.py:45:8: W0612: Unused variable 'old_position' (unused-variable)

moneypoly/player.py:1:0: W0611: Unused import sys (unused-import)

Your code has been rated at 7.92/10 (previous run: 7.92/10, +0.00)

- Iteration 48: Added a final newline at end of file to fix `moneypoly/player.py:87:0: C0304: Final newline missing (missing-final-newline)`

- Iteration 49: Added module level docstring to fix `moneypoly/player.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

"""Player model and movement/economy helpers for MoneyPoly."""

- Iteration 50: Changed absolute import to relative import to fix `moneypoly/player.py:4:0: E0401: Unable to import 'moneypoly.config' (import-error)`

from config import STARTING_BALANCE, BOARD_SIZE, GO_SALARY, JAIL_POSITION

- Iteration 51: Reduced `Player` instance attributes by making `is_eliminated` a class-level default, fixing `moneypoly/player.py:7:0: R0902: Too many instance attributes (8/7) (too-many-instance-attributes)`

class Player:

  is_eliminated = False

- Iteration 52: Removed the unused `old_position` variable in `move()` to fix `moneypoly/player.py:47:8: W0612: Unused variable 'old_position' (unused-variable)`

self.position = (self.position + steps) % BOARD_SIZE

- Iteration 53: Removed unused import `sys` to fix `moneypoly/player.py:3:0: W0611: Unused import sys (unused-import)`

from config import STARTING_BALANCE, BOARD_SIZE, GO_SALARY, JAIL_POSITION

game.py

Initial Pylint Output:

pylint moneypoly/game.py

********** Module game

moneypoly/game.py:453:0: C0325: Unnecessary parens after 'not' keyword (superfluous-parens)

moneypoly/game.py:462:0: C0325: Unnecessary parens after 'not' keyword (superfluous-parens)

moneypoly/game.py:468:0: C0304: Final newline missing (missing-final-newline)

moneypoly/game.py:1:0: C0114: Missing module docstring (missing-module-docstring)

moneypoly/game.py:3:0: E0401: Unable to import 'moneypoly.config' (import-error)

moneypoly/game.py:12:0: E0401: Unable to import 'moneypoly.player' (import-error)

moneypoly/game.py:13:0: E0401: Unable to import 'moneypoly.board' (import-error)

moneypoly/game.py:14:0: E0401: Unable to import 'moneypoly.bank' (import-error)

moneypoly/game.py:15:0: E0401: Unable to import 'moneypoly.dice' (import-error)

moneypoly/game.py:16:0: E0401: Unable to import 'moneypoly.cards' (import-error)

moneypoly/game.py:17:0: E0401: Unable to import 'moneypoly' (import-error)

moneypoly/game.py:20:0: R0902: Too many instance attributes (9/7) (too-many-instance-attributes)

moneypoly/game.py:295:4: R0912: Too many branches (15/12) (too-many-branches)

moneypoly/game.py:381:28: W1309: Using an f-string that does not have any interpolated variables (f-string-without-interpolation)

moneypoly/game.py:402:12: R1723: Unnecessary "elif" after "break", remove the leading "el" from "elif" (no-else-break)

moneypoly/game.py:1:0: W0611: Unused import os (unused-import)

moneypoly/game.py:3:0: W0611: Unused GO_TO_JAIL_POSITION imported from moneypoly.config (unused-import)

Your code has been rated at 8.70/10 (previous run: 8.70/10, +0.00)

- Iteration 54: Removed unnecessary parentheses after `not` for trade partner index validation to fix `moneypoly/game.py:453:0: C0325: Unnecessary parens after 'not' keyword (superfluous-parens)`

```
if not 0 <= idx < len(others):
    return
```

- Iteration 55: Removed unnecessary parentheses after `not` for property index validation to fix `moneypoly/game.py:462:0: C0325: Unnecessary parens after 'not' keyword (superfluous-parens)`

if not 0 <= pidx < len(player.properties):

return

- Iteration 56: Added module level docstring to fix `moneypoly/game.py:1:0: C0114: Missing module docstring (missing-module-docstring)`

"""Core game loop and turn-resolution logic for MoneyPoly."""

- Iteration 57: Changed absolute config import to relative import to fix `moneypoly/game.py:5:0: E0401: Unable to import 'moneypoly.config' (import-error)`

from config import (

- Iteration 58: Changed absolute player import to relative import to fix `moneypoly/game.py:14:0: E0401: Unable to import 'moneypoly.player' (import-error)`

from player import Player

- Iteration 59: Changed absolute board import to relative import to fix `moneypoly/game.py:15:0: E0401: Unable to import 'moneypoly.board' (import-error)`

from board import Board

- Iteration 60: Changed absolute bank import to relative import to fix `moneypoly/game.py:16:0: E0401: Unable to import 'moneypoly.bank' (import-error)`

from bank import Bank

- Iteration 61: Changed absolute dice import to relative import to fix `moneypoly/game.py:17:0: E0401: Unable to import 'moneypoly.dice' (import-error)`

from dice import Dice

- Iteration 62: Changed absolute cards import to relative import to fix `moneypoly/game.py:18:0: E0401: Unable to import 'moneypoly.cards' (import-error)`

from cards import CardDeck, CHANCE_CARDS, COMMUNITY_CHEST_CARDS

- Iteration 63: Changed package-level `ui` import to local module import to fix `moneypoly/game.py:19:0: E0401: Unable to import 'moneypoly' (import-error)`

import ui

- Iteration 64: Reduced `Game` instance attributes by replacing separate deck fields with a single `decks` mapping and removing unused `running` state, fixing `moneypoly/game.py:22:0: R0902: Too many instance attributes (9/7) (too-many-instance-attributes)`

self.decks = {

"chance": CardDeck(CHANCE_CARDS),

"community_chest": CardDeck(COMMUNITY_CHEST_CARDS),

}

- Iteration 65: Refactored `_apply_card` to use action dispatch helpers, reducing branch complexity and fixing `moneypoly/game.py:298:4: R0912: Too many branches (15/12) (too-many-branches)`

handlers = {

"collect": self._card_collect,

"pay": self._card_pay,

"jail": self._card_jail,

}

- Iteration 66: Removed unnecessary f-string wrapper for a static string to fix `moneypoly/game.py:395:28: W1309: Using an f-string that does not have any interpolated variables (f-string-without-interpolation)`

ui.print_banner("GAME OVER")

- Iteration 67: Removed unnecessary `elif` after `break` in the pre-roll menu to fix `moneypoly/game.py:416:12: R1723: Unnecessary "elif" after "break" (no-else-break)`

if choice == 0:

break

if choice == 1:

ui.print_standings(self.players)

- Iteration 68: Removed unused `os` import to fix `moneypoly/game.py:3:0: W0611: Unused import os (unused-import)`

- Iteration 69: Removed unused `GO_TO_JAIL_POSITION` from config imports to fix `moneypoly/game.py:3:0: W0611: Unused GO_TO_JAIL_POSITION imported from config (unused-import)`

1.3 White Box Test Cases

Error #	White-box test case	Why this test is needed (simple)	Error / logical issue found	Fix made
1	`tests/test_white_box_cases.py::TestPackageImports.test_import_game_from_package`	The code has many import decisions that depend on how modules are loaded. This test checks the same package path used by `main.py`.	Importing `moneypoly.game` failed with `ModuleNotFoundError: No module named 'config'` because internal modules used non-package imports (`from config import ...`, `from property import ...`).	Updated `game.py`, `player.py`, `board.py`, and `bank.py` to use package-relative imports with a direct-execution fallback (`try: from .x import ... except ImportError: from x import ...`).

Error #	White-box test case	Why this test is needed (simple)	Error / logical issue found	Fix made
2	`tests/test_white_box_cases.py::TestDiceBranches.test_roll_allows_six_as_edge_value`	Dice roll has a boundary branch (maximum face value). Edge value `6` must be reachable for correct gameplay probabilities.	`Dice.roll()` used `random.randint(1, 5)`, so face value `6` was impossible.	Changed dice bounds to `random.randint(1, 6)` for both dice in `moneypoly/dice.py`.
3	`tests/test_white_box_cases.py::TestPlayerMovementBranches.test_passing_go_without_landing_also_collects_salary`	Player movement has two money branches: landing on Go and passing Go. Both must credit salary.	`Player.move()` only paid salary when `position == 0`, so passing Go to a non-zero tile paid nothing.	Updated `Player.move()` to detect wrap-around (`new_position < old_position`) and pay `GO_SALARY` for both passing and landing on Go.
4	`tests/test_white_box_cases.py::TestPropertyOwnershipLogic.test_all_owned_by_requires_every_property`	Monopoly group logic depends on a complete set. This branch test checks the partial-ownership path.	`PropertyGroup.all_owned_by()` used `any(...)`, so even one owned property incorrectly counted as full ownership.	Replaced `any(...)` with `all(...)` in `moneypoly/property.py`; also added tests for true/false/`None` ownership states.
5	`tests/test_white_box_cases.py::TestGameFinancialLogic.test_buy_property_with_exact_balance_succeeds`	Affordability has a boundary branch (`balance == price`) that must allow purchase.	`buy_property()` rejected exact-balance purchases because condition was `balance <= price`.	Changed affordability check to `balance < price` in `moneypoly/game.py`.
6	`tests/test_white_box_cases.py::TestGameFinancialLogic.test_pay_rent_transfers_to_owner`	Rent payment has two required state updates: renter decreases and owner increases. This test checks both sides.	`pay_rent()` deducted tenant money but did not add rent to the property owner.	Added `prop.owner.add_money(rent)` in `moneypoly/game.py` after tenant deduction.
7	`tests/test_white_box_cases.py::TestGameFinancialLogic.test_jail_fine_choice_deducts_player_balance`	Jail-turn flow has a decision branch when player chooses to pay a fine. This must update both bank and player balance.	In voluntary jail release path, bank collected the fine but player balance was not reduced.	Added `player.deduct_money(JAIL_FINE)` before bank collection in `_handle_jail_turn()` (`moneypoly/game.py`).

Error #	White-box test case	Why this test is needed (simple)	Error / logical issue found	Fix made
8	`tests/test_white_box_cases.py::TestGameFinancialLogic.test_find_winner_returns_highest_net_worth`	End-game decision must select the highest net worth branch, otherwise game result is reversed.	`find_winner()` returned `min(...)`, so the poorest player was declared winner.	Replaced `min(...)` with `max(...)` in `moneypoly/game.py`.
9	`tests/test_white_box_cases.py::TestGameDeepBranchCoverage.test_unmortgage_insufficient_funds_keeps_property_mortgaged`	Unmortgage has a failure branch where the player cannot afford the cost; property state must remain mortgaged in that case.	`unmortgage_property()` called `prop.unmortgage()` before affordability check, so mortgage flag was cleared even when the action failed.	Reordered logic in `moneypoly/game.py` to validate mortgaged state and affordability first, then call `prop.unmortgage()` only on success.
10	`tests/test_white_box_cases.py::TestGameDeepBranchCoverage.test_trade_transfers_cash_from_buyer_to_seller`	Trade must move both ownership and money; otherwise seller loses a property for free.	`trade()` deducted money from buyer but never credited seller, causing value loss.	Updated `trade()` in `moneypoly/game.py` to add cash to seller and to reject non-positive cash trades.
11	`tests/test_white_box_cases.py::TestGameDeepBranchCoverage.test_buy_property_rejects_already_owned_property`	`buy_property()` is public and can be called directly, so it must guard against buying an already-owned tile.	`buy_property()` only checked balance and allowed overwriting `owner`, enabling illegal direct purchases.	Added an owner check in `buy_property()` (`moneypoly/game.py`) that returns `False` if the property is already owned.
12	`tests/test_white_box_cases.py::TestBankAndDeckBranches.test_bank_give_loan_non_positive_and_positive` and `...test_bank_give_loan_raises_when_insufficient_funds`	Loan flow should maintain bank accounting invariants and prevent over-lending.	`give_loan()` credited player but did not reduce bank funds, and it had no guard against issuing more than reserves.	Updated `moneypoly/bank.py` so loans reduce `_funds` and raise `ValueError` when requested loan exceeds available reserves.
13	`tests/test_white_box_cases.py::TestBankAndDeckBranches.test_card_deck_em`	Empty decks should be safe to introspect; utility methods should not crash game/reporting paths.	`cards_remaining()` and `__repr__()` used modulo by `len(cards)`, raising	Added empty-deck guards in `moneypoly/cards.py`: `cards_remaining()` now returns `0`, and `__repr__()`

Error #	White-box test case	Why this test is needed (simple)	Error / logical issue found	Fix made
	pty_remaining_and_repr_are_safe		ZeroDivisionError when the deck was empty.	returns CardDeck(0 cards, next=none).
14	tests/test_white_box_cases.py::TestPlayerPropertyAndBoardExtras.test_player_net_worth_includes_property_values	Winner logic depends on net_worth(). If property values are ignored, end-game ranking becomes incorrect.	Player.net_worth() returned only cash balance and ignored all owned properties.	Updated Player.net_worth() in moneypoly/player.py to include owned property values (full price for normal properties, mortgage value for mortgaged ones).
15	tests/test_white_box_cases.py::TestGameDeepBranchCoverage.test_auction_property_rejects_already_owned_property	Auction API should be safe even if called directly with invalid state. Owned properties must not be re-auctioned.	auction_property() allowed bidding on an already-owned property, deducted money, and effectively let owner buy their own tile again.	Added an early owner guard in moneypoly/game.py so auction_property() exits immediately when prop.owner is not None.
16	tests/test_white_box_cases.py::TestGameDeepBranchCoverage.test_interactive_menu_loan_over_limit_does_not_crash	Interactive input should be fault-tolerant. Invalid loan requests should be handled without terminating the game loop.	After adding over-limit checks in Bank.give_loan(), the pre-roll menu propagated ValueError and crashed when a player requested more than bank reserves.	Wrapped the loan call in interactive_menu() (moneypoly/game.py) with try/except ValueError and displayed a user-friendly denial message instead of crashing.
17	tests/test_white_box_cases.py::TestGameDeepBranchCoverage.test_card_collect_with_insufficient_bank_funds_is_safe	Card resolution should be robust under depleted-bank states and must not terminate gameplay unexpectedly.	_card_collect() directly called bank.pay_out() and propagated ValueError when the bank had insufficient funds.	Updated _card_collect() in moneypoly/game.py to catch ValueError, log a clear message, and treat payout as $0 so turn flow continues safely.
18	tests/test_white_box_cases.py::TestMainModuleBranches.test_main_rejects_less_than_two_players	Entry flow must enforce the documented minimum-two-players precondition to avoid invalid game setup.	main() accepted one player and still created/ran the game, despite prompt text stating minimum 2 players.	Added validation in whitebox/moneypoly/main.py to raise ValueError when fewer than 2 names are provided, reusing existing setup-error handling.
19	tests/test_white_box_cases.py::TestGameDeepBranchCoverage.test_apply_car	Card-handling should be resilient to malformed deck data and	_apply_card() indexed card["action"] and card["value"] directly, raising	Updated _apply_card() in moneypoly/game.py to read fields safely via .get(), default

Error #	White-box test case	Why this test is needed (simple)	Error / logical issue found	Fix made
	`d_and_card_helpers_branches` (malformed card path)	should never crash a running game on bad card payloads.	`KeyError` for malformed cards missing required keys.	value to `0`, and ignore cards with missing action using a clear message.

Passing Test Cases (No Bug Found) and Why Needed

Test case	Why this test is needed	Result
`TestDiceBranches.test_doubles_streak_increments_then_resets`	Verifies doubles-state branch resets correctly after a non-double roll, which affects jail logic.	Passed; streak handling is correct.
`TestPlayerMovementBranches.test_landing_on_go_collects_salary`	Confirms the landing-on-Go salary path works, not just pass-Go.	Passed; salary credited correctly.
`TestPropertyOwnershipLogic.test_all_owned_by_true_when_every_property_matches`	Checks the positive branch for full-group ownership after fixing monopoly logic.	Passed; full ownership recognized.
`TestPropertyOwnershipLogic.test_all_owned_by_none_player_returns_false`	Guards against invalid state where `None` is treated as a valid owner.	Passed; safely returns `False`.
`TestBoardBranchCoverage.test_is_purchasable_branches`	Covers all decision branches in `is_purchasable`: non-property, available, mortgaged, already owned.	Passed; branch behavior is consistent.
`TestBankAndDeckBranches.test_bank_collect_positive_and_negative_amounts`	Validates bank state changes for both positive and negative collection values.	Passed; balance updates as implemented.
`TestBankAndDeckBranches.test_bank_pay_out_branches`	Covers zero, negative, and valid payout branches.	Passed; payout guards work.

Test case	Why this test is needed	Result
`TestBankAndDeckBranches.test_bank_pay_out_raises_when_insufficient_funds`	Ensures insufficient-funds path raises error instead of silently corrupting state.	Passed; `ValueError` raised.
`TestBankAndDeckBranches.test_bank_give_loan_non_positive_and_positive`	Verifies loan edge cases (`<=0`) and valid-loan state updates (balance, count, total).	Passed; loan logic is consistent.
`TestBankAndDeckBranches.test_bank_summary_prints`	Confirms reporting function executes without runtime errors.	Passed; summary prints correctly.
`TestBankAndDeckBranches.test_card_deck_draw_empty_and_cycle`	Tests empty-deck branch and wrap-around draw behavior for long games.	Passed; empty returns `None`, cycle works.
`TestBankAndDeckBranches.test_card_deck_peek_and_reshuffle`	Checks that `peek` is non-destructive and reshuffle resets deck index.	Passed; state transitions are correct.
`TestBankAndDeckBranches.test_card_deck_empty_remaining_and_repr_are_safe`	Verifies empty-deck helper methods are safe and return stable values instead of crashing.	Passed; empty deck now reports `0` remaining and a safe repr string.
`TestPlayerPropertyAndBoardExtras.test_player_negative_money_operations_raise`	Validates input guards against invalid negative amount operations.	Passed; exceptions raised correctly.
`TestPlayerPropertyAndBoardExtras.test_player_non_wrap_move_and_jail_and_property_helpers`	Covers non-wrap movement, jail transition, and property add/remove helper paths.	Passed; all helper states update correctly.
`TestPlayerPropertyAndBoardExtras.test_player_bankruptcy_threshold`	Checks bankruptcy threshold boundary (`0` and above `0`).	Passed; threshold behavior is correct.

Test case	Why this test is needed	Result
`TestPlayerPropertyAndBoardExtras.test_property_constructor_argument_validation`	Verifies defensive branches for invalid constructor argument combinations.	Passed; invalid usage raises `TypeError`.
`TestPlayerPropertyAndBoardExtras.test_property_rent_mortgage_and_availability_branches`	Covers rent branches (normal, monopoly, mortgaged) and mortgage/unmortgage transitions.	Passed; values and flags are correct.
`TestPlayerPropertyAndBoardExtras.test_property_group_helpers`	Verifies group helper methods (`add_property`, `size`, owner count) state integrity.	Passed; helper behavior is stable.
`TestPlayerPropertyAndBoardExtras.test_board_helpers`	Covers board helper methods (`tile type`, `special`, owner listing, unowned listing).	Passed; helper queries are consistent.
`TestGameDeepBranchCoverage.test_advance_turn_wraps_and_increments_counter`	Validates turn-index wrap-around and turn counter increments together.	Passed; turn progression is correct.
`TestGameDeepBranchCoverage.test_play_turn_branch_in_jail`	Covers jailed-player branch in turn flow.	Passed; jail handler + turn advance invoked.
`TestGameDeepBranchCoverage.test_play_turn_branch_three_doubles_sends_to_jail`	Checks penalty branch for triple doubles.	Passed; player sent to jail correctly.
`TestGameDeepBranchCoverage.test_play_turn_branch_doubles_gets_extra_turn`	Verifies extra-turn branch when roll is doubles.	Passed; no turn advance occurs.
`TestGameDeepBranchCoverage.test_play_turn_branch_normal_advance`	Confirms normal turn path advances correctly when not doubles.	Passed; standard flow works.

Test case	Why this test is needed	Result
`TestGameDeepBranchCoverage.test_move_and_resolve_tile_branches`	Covers tile decision branches: income tax, luxury tax, go-to-jail, free parking.	Passed; each tile branch behaves correctly.
`TestGameDeepBranchCoverage.test_move_and_resolve_card_and_property_branches`	Covers chance/chest/property/railroad resolution and related handler calls.	Passed; dispatch paths are correct.
`TestGameDeepBranchCoverage.test_handle_property_tile_branches`	Covers property interaction choices (`buy`, `auction`, `skip`, own tile, opponent tile).	Passed; correct branch handlers are called.
`TestGameDeepBranchCoverage.test_buy_rent_mortgage_unmortgage_trade_branches`	Stress-tests financial branches and failure/success transitions in core property economy actions.	Passed; action guards and state changes are correct.
`TestGameDeepBranchCoverage.test_auction_property_winner_and_no_bid_paths`	Covers auction invalid/pass and winning-bid paths.	Passed; winner/no-winner outcomes are correct.
`TestGameDeepBranchCoverage.test_handle_jail_turn_branches`	Covers all jail-turn branches: card use, voluntary fine, wait, mandatory release.	Passed; each branch updates state correctly.
`TestGameDeepBranchCoverage.test_apply_card_and_card_helpers_branches`	Covers card dispatch edge cases and helper action effects (collect/pay/jail/move/collect-from-others).	Passed; card effects apply correctly.
`TestGameDeepBranchCoverage.test_check_bankruptcy_and_find_winner_empty`	Covers elimination cleanup branch and empty-player winner branch.	Passed; cleanup and `None` winner behavior are correct.
`TestGameDeepBranchCoverage.test_run_branches`	Covers game-loop exit conditions (single player and no players).	Passed; loop exits safely.

Test case	Why this test is needed	Result
`TestGameDeepBranchCoverage.test_interactive_menu_and_submenus`	Covers menu routing and invalid selection branches in mortgage/unmortgage/trade flows.	Passed; menu control flow is correct.
`TestUIHelpers.test_safe_int_input_valid`	Checks normal integer parsing path.	Passed; input converted correctly.
`TestUIHelpers.test_safe_int_input_invalid_returns_default`	Checks invalid-input fallback path to avoid crashes.	Passed; default returned correctly.
`TestUIHelpers.test_confirm_yes`	Verifies positive confirmation path.	Passed; returns `True`.
`TestUIHelpers.test_confirm_no`	Verifies negative confirmation path.	Passed; returns `False`.
`TestUIHelpers.test_format_currency`	Confirms currency formatting helper for user-facing output.	Passed; formatting is correct.
`TestMainModuleBranches.test_get_player_names_parsing`	Validates input normalization for comma-separated names and empty-fragment filtering.	Passed; parsing returns clean player-name list.
`TestMainModuleBranches.test_main_runs_game_successfully`	Confirms happy-path entry flow creates `Game` and executes `run()`.	Passed; entry flow works as expected.
`TestMainModuleBranches.test_main_handles_keyboard_interrupt`	Ensures graceful exit branch is handled without crashing user session.	Passed; interruption is handled cleanly.
`TestMainModuleBranches.test_main_handles_value_error`	Verifies setup-error path is surfaced as a friendly message.	Passed; setup errors are reported correctly.
`TestMainModuleBranches.test_main_module_guard_executes`	Checks `if __name__ == "__main__"` execution path for script usage.	Passed; module guard path executes correctly.

Test case	Why this test is needed	Result
`TestDiceBranches.test_dice_repr`	Confirms dice debug representation is stable and informative for diagnostics.	Passed; repr contains expected dice metadata.
`TestBankAndDeckBranches.test_bank_repr`	Verifies bank object representation is safe and useful for state tracing.	Passed; repr generation is correct.
`TestGameDeepBranchCoverage.test_move_and_resolve_railroad_and_property_none_variants`	Covers mixed property-resolution variants to ensure handler dispatch remains correct.	Passed; dispatch behavior is consistent.
`TestGameDeepBranchCoverage.test_move_and_resolve_blank_tile_path`	Validates neutral-tile path still runs bankruptcy checks and continues flow.	Passed; blank-tile branch behaves correctly.
`TestGameDeepBranchCoverage.test_run_executes_loop_body`	Confirms run-loop body executes at least one turn under boundary turn settings.	Passed; loop body and standings output are triggered.
`TestGameDeepBranchCoverage.test_interactive_menu_loan_zero_loops_without_loan`	Ensures zero-loan input does not issue unintended loans in menu flow.	Passed; no loan issued for zero amount.
`TestGameDeepBranchCoverage.test_interactive_menu_invalid_choice_falls_through`	Confirms out-of-range pre-roll menu choices do not corrupt game state.	Passed; invalid choices are safely ignored.